Case Study Report

Design and Creative Technologies

Torrens University, Australia

Student: Luis Guilherme de Barros Andrade Faria - A00187785

Subject Code: CCF 501

Subject Name: Cloud Computing Fundamentals

Assessment No.: 2

Title of Assessment: Case Study Report

Lecturer: Dr. Dyvia Leekha

Date: April 2026

Copyright © 2026 by Luis G B A Faria

Permission is hereby granted to make and distribute verbatim copies of this document provided the copyright notice and this permission notice are preserved on all copies.

Table of Contents

1. Introduction 3
2. Case Study 1 – Victoria Police & Microsoft Power Platform 4
2.1. Case Overview 4
2.2. Service Model Analysis 5
2.3. Deployment Model Comparison 6
2.4. Cloud Services Analysis 7
2.5. Reflection 10
3. Case Study 2 – ChatAndBuild & Google Cloud 10
3.1. Case Overview 11
3.2. Service Model Analysis 11
3.3. Deployment Model Comparison 12
3.4. Deployment Model Comparison 13
3.5. Reflection 15
4. Case Study 3 – Voith & AWS 16
5. Conclusion 21
6. Appendices 23
6.1. Appendix A – Cloud vs Traditional IT Comparison Table 23
6.2. Appendix G – Glossary of Terms 23
7. References 25

Cloud Services and Deployment Models: A Cross-Sector Case Study Analysis

1. Introduction

Cloud computing has fundamentally reshaped how organizations across every sector deliver services, manage infrastructure, and drive automation. Rather than owning and operating physical data centers, enterprises now consume compute, storage, networking, and intelligence on demand – scaling elasticity, paying only for what they use, and offloading infrastructure management to specialist providers (Mell & Grance, 2011). The National Institute of Standards and Technology (NIST) defines cloud computing as a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort (Mell & Grance, 2011). The shift from Capital Expenditure (CAPEX)-heavy on-premises environments to Operational Expenditure (OPEX)-driven cloud models is no longer a trend exclusive to technology companies, it is a cross-industry imperative that spans government agencies, AI-native startups, and century-old industrial manufacturers alike (McHaney, 2021).

This report analyses three real-world case studies drawn from distinct sectors – government law enforcement (Victoria Police), artificial intelligence and software startup (ChatAndBuild), and industrial manufacturing (Voith) – each provided by the subject facilitator and representing a different combination of cloud service model, deployment model, and provider ecosystem. By examining how cloud was applied in each context, this report aims to: describe the essential cloud elements that enabled business automation in each case; distinguish cloud adoption from

traditional IT infrastructure approaches and identify the key cloud service offerings that made each outcome possible.

The three cases collectively illustrate how cloud computing is not a single, monolithic technology but a spectrum of services and deployment strategies calibrated to organizational context, risk appetite, and automation goals. Victoria Police leveraged Microsoft Power Platform (PaaS) within a hybrid multi-environment model to democratize application development across a law enforcement agency – transforming previously invisible, ungoverned Microsoft Access databases into a governed, scalable innovation ecosystem (Microsoft, 2026). ChatAndBuild, an AI-native startup based in Singapore, migrated to Google Cloud Platform to eliminate the manual infrastructure burden that was impeding real-time AI agent creation, achieving a 45% reduction in infrastructure overhead and a 30-70% improvement in generation latency (Google Cloud, n.d.). Voith, a 150-year-old manufacturing company operating across 60 countries, selected AWS following a competitive provider assessment to consolidate 3,500 servers distributed across 146 global locations into six AWS regions – projecting 30% cost savings compared to the legacy setup (Amazon Web Services, 2026).

2. Case Study 1 – Victoria Police & Microsoft Power Platform

2.1. Case Overview

Victoria Police – Australia's principal state law enforcement agency – adopted Microsoft Power Platform to establish a **fusion development framework**: a governed model that enables non-professional developers (police personnel across business units) to build data-driven applications safely, without depending solely on a constrained central IT team

(Microsoft, 2026). The catalyst was a surge in self-directed application-building activity during the pandemic, when employees teaching themselves to use Power Apps revealed both the appetite for low-code tooling and the security risk of ungoverned 'shadow' applications built on standalone Microsoft Access databases. Rather than restricting this behavior, Victoria Police channeled it – creating structured pathways from personal productivity through to enterprise-grade production deployment, with two applications (Global Whereabouts and Bail Reporting) already progressing to enterprise adoption (Microsoft, 2026).

Figure 1 – ABC Enterprise cloud traffic flow - from user request through Route 53, load balancing, compute, storage, and database layers

2.2. Service Model Analysis

Victoria Police's cloud adoption aligns primarily with a **Platform as a Service (PaaS)** model. Power Apps and Power Platform abstract the underlying infrastructure – Microsoft manages the compute, networking, and database layers of Dataverse and the Power Platform runtime – while Victoria Police's developers focus on application logic, data models, and user experience (McHaney, 2021). This is the defining characteristic of PaaS: the provider controls the platform, the customer controls the application (IBM, n.d.0a). Some Microsoft 365 productivity workloads (SharePoint, Teams, Outlook) sit at the **SaaS** layer – fully managed software delivered over the network. There is no IaaS footprint in this case; Victoria Police has not provisioned raw virtual machines or unmanaged storage – all development occurs within the abstracted Power Platform environment (Microsoft, 2026).

Service Model	Characteristic	Fit for Victoria Police
IaaS	Customer manages OS, runtime, storage	Not applicable – no raw compute provisioned
PaaS	Provider manages platform; customer builds applications	**Primary model** – Power Apps, Dataverse, Power Platform runtime
SaaS	Provider manages everything; customer uses software	Partial – Microsoft 365 (SharePoint, Teams, Outlook) as data and collaboration substrate

Table 1 – Service model analysis for Victoria Police (Microsoft, 2026).

2.3. Deployment Model Comparison

Victoria Police operates a **hybrid, multi-environment deployment model**: a personal productivity environment (the default Microsoft 365 tenant available to all staff), dedicated fusion environments (isolated, governed Power Platform environments provisioned on request by business units with executive endorsement), and enterprise production environments (reviewed and approved by the central Power Platform team) (Microsoft, 2026). This tiered model is a practical application of hybrid cloud architecture – multiple environment tiers serving different governance, security, and scale requirements simultaneously (Manvi & Shyam, 2021).

Deployment Model	Governance	Security	Scalability	Fit for Victoria Police
On-premises (Microsoft Access)	Low – invisible to IT	High risk – unadited shadow apps	None	Prior state – security blind spots, no transparency
Hybrid (Multi-env Power Platform)	High – tiered approval flow with exec endorsement	High – centralized standards, security review	Moderate	**Adopted** – balances innovation with enterprise controls

Public Cloud (ungoverned)	Low	Low	High	Creates same shadow IT risk as pre-migration state
Private Cloud	High	High	Limited	High cost, no low-code developer ecosystem available

Table 2 – Deployment model comparison for Victoria Police (Microsoft, 2026; Manvi & Shyam, 2021).

2.4. Cloud Services Analysis

The fusion framework relies on a layered set of Microsoft Power Platform services, each enabling a distinct stage of the application lifecycle (Microsoft, 2026):

Service	Purpose	Business Outcome
Power Apps (Low-code / Paas)	Enable non-professional developers to build data-driven apps without code	Personnel build and iterate solutions without the central IT bottleneck
Microsoft Dataverse	Model-driven, structure data store underpinning enterprise apps	Replaces isolated MS Access databases with a governed, transparent and auditable data layer
Power Platform Environments	Isolated development, test, and production workspaces per business unit	Clear pathway from personal innovation to enterprise-grade development
Power Platform Approval Flow	Governs requests for dedicated fusion environments via a model-driven app	Ensures executive endorsement and security team review before promotion to production
Microsoft 365 (SharePoint)	Data substrate and collaboration layer for initial prototypes	Familiar toolset lowers the barrier to entry for non-technical builders

Copilot Studio (exploration)	AI-assisted app development using generative coding	Future capability to further accelerate the build-to-deploy cycle

Table 3 – Microsoft Power Platform services utilised by Victoria Police and their business outcomes (Microsoft, 2026).

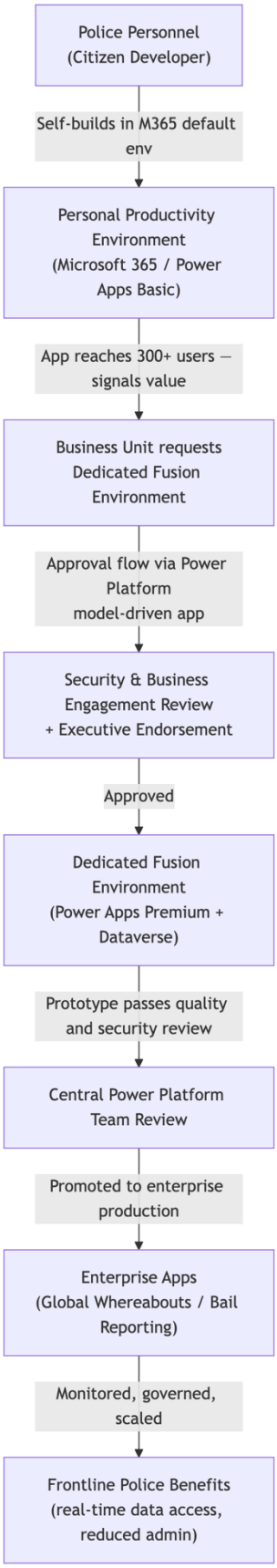
Police Personnel (Citizen Developer)
Self-builds in M365 default env
Personal Productivity Environment (Microsoft 365 / Power Apps Basic)
App reaches 300+ users — signals value
Business Unit requests Dedicated Fusion Environment
Approval flow via Power Platform model-driven app
Security & Business Engagement Review + Executive Endorsement
Approved
Dedicated Fusion Environment (Power Apps Premium + Dataverse)
Prototype passes quality and security review
Central Power Platform Team Review
Promoted to enterprise production
Enterprise Apps (Global Whereabouts / Bail Reporting)
Monitored, governed, scaled
Frontline Police Benefits (real-time data access, reduced admin)

Figure 1: Victoria Police fusion development lifecycle: from citizen developer prototype through tiered governance to enterprise production (Microsoft, 2026).

2.5. Reflection

The Victoria Police case demonstrates how cloud's **on-demand self-service** characteristic (Mell & Grance, 2011) can be safely extended to non-professional developers when paired with tiered governance. In a traditional IT model, every application request would queue behind the central IT team – a bottleneck that drove staff to build ungoverned Microsoft Access solutions outside the organization's visibility. The Power Platform model inverts this dynamic: rather than centralizing all development capacity, it governs the pathway to production while allowing distributed development (Microsoft, 2026). This represents a fundamental distinction from traditional IT: the cloud enables a spectrum of developer access that on-premises tooling structurally cannot support (McHaney, 2021).

However, risks remain significant. Sensitive law enforcement data held within a cloud-managed PaaS environment requires robust data residency controls and compliance with the Australian Protective Security Policy Framework (PSPF). As enterprise adoption scales to cover regions and thousands of officers, the risk surface expands correspondingly – a concern Efrain Tionko (Head of Digital Delivery and Transformation, Victoria Police) explicitly acknowledged when noting the need to increase controls as employee access grows (Microsoft, 2026).

3. Case Study 2 – ChatAndBuild & Google Cloud

3.1. Case Overview

ChatAndBuild is a Singapore-based AI-native startup whose platform enables anyone, from a 4-year-old designing a mathematics game to a 78-year-old building a strategy simulation, to turn natural language into working applications and AI agents instantly (Google Cloud, n.d.). Serving over 140,000 users, the company's vision of the "Intelligence-Ownership Economy" – where users own Non-Fungible Agents (NFAs) with persistent memory, skills, and personality – placed extreme demands on infrastructure reliability and latency. The company's previous infrastructure imposed 30–70% higher latency on real-time generation, caused context rot in long-lived agents (loss of user history), and required engineers to manually provision servers days before global hackathons to prevent IP surge crashes (Google Cloud, n.d.). ChatAndBuild migrated to Google Cloud to establish a fully AI-native, elastically scalable infrastructure capable of operating at global speed.

3.2. Service Model Analysis

ChatAndBuild's architecture spans **IaaS** and **PaaS** layers. Google Compute Engine provides the raw virtual machine infrastructure (IaaS) that enables elastic scaling of the worker fleet from approximately 10 to 100+ instances during global hackathon events, handled automatically through instance groups and autoscaling policies (Google Cloud, n.d.). On top of this, a rich suite of managed PaaS services handles AI orchestration, containerized workload management, data analytics, state synchronization, and CI/CD (Continuous Integration / Continuous Development) – abstracting operational complexity so that the

engineering team can focus on product development rather than infrastructure management (McHaney, 2021).

Service Model	Characteristic	Fit for Victoria Police
IaaS	Raw compute provisioned on demand; customer manages OS and runtime	Google Compute Engine – elastic worker fleetr scaling for burst demand
PaaS	Managed platform; provider handles runtime and infrastructure operations	**Primary layer** – Vertex AI, GKE, Cloud Run, BigQuery, Cloud SQL, Firebase
SaaS	Fully managed software	Partial – Microsoft 365 (SharePoint, Teams, Outlook) as data and collaboration substrate

Table 4 – Service model analysis for ChatAndBuild (Google Cloud, n.d.).

3.3. Deployment Model Comparison

ChatAndBuild operates on **public cloud** across multiple Google Cloud regions. Cloud Load Balancing routes each request to the nearest available region, ensuring sub-second response times for users globally. The multi-region public cloud model was the only viable option for a startup serving over 140,000 users across time zones, where global hackathon spikes require instantly elastic capacity – and where the prior on-premises approach required engineers to manually provision servers days in advance (Google Cloud, n.d.).

Deployment Model	Elasticity	Global Reach	Latency	Fit for Victoria Police
On-premises	None – fixed servers	Single geography	High	Prior state – manual-provisioning; 30-70% higher latency; engineers as *"human autoscalers"*

Private Cloud	Limited	Single geography	Medium	No global edge; cannot serve 140,000+ users across time zones
Public Cloud (Multi-region)	High – Auto-scale 10→100+ instances in minutes	Global PoP via Cloud Load Balancing	Low – sub-second	**Adopted** – GKE + Cloud Run auto-scale to demand with zero downtime
Multi-Cloud	Highest	Highest	Lowest	Premature complexity for current startup scale

Table 5: Deployment model comparison for ChatAndBuild (Google Cloud, n.d.; Manvi & Shyam, 2021).

3.4. Deployment Model Comparison

The Google Cloud architecture forms an integrated "idea-to-app" engine where each service layer contributes to the real-time generation pipeline (Google Cloud, n.d.):

Service	Purpose	Business Outcome
Vertex AI + Gemini models	Routes prompts to specialised coding and research agents; multimodal generation	Core AI engine – 30–70% latency reduction vs previous setup
Google Kubernetes Engine (GKE)	Isolated namespaces per agent graph; manages thousands of concurrent builder sessions	Zero-downtime event scaling – 13B+ tokens processed at a global hackathon
Cloud Run + Cloud Functions	Stateless microservices that auto-spin for traffic bursts	Handles demand spikes without idle compute; no pre-provisioning required
Cloud Load Balancing	Routes every request to the nearest region	Sub-second response times globally

BigQuery	Telemetry layer – tracks billions of tokens to prevent context rot	Maintains long-term agent memory continuity and intelligence
Cloud SQL + Firestore + Firebase	Structured data management and real-time user state synchronization	Eliminates context rot; persistent, coherent agent sessions
Cloud CDN + Cloud Storage	Stores and delivers generated application assets globally	Final app delivery to end users with low latency
Cloud Build + Artifact Registry	CI/CD pipeline for continuous feature deployment	Feature delivery velocity doubled (2× faster)
VPC Service Controls + Cloud Armor + IAM	Security perimeter for regulated enterprise and government markets	SOC2 compliance; strict audit logging for enterprise and government partners

Table 6: Google Cloud services utilized by ChatAndBuild and their business outcomes (Google Cloud, n.d.).

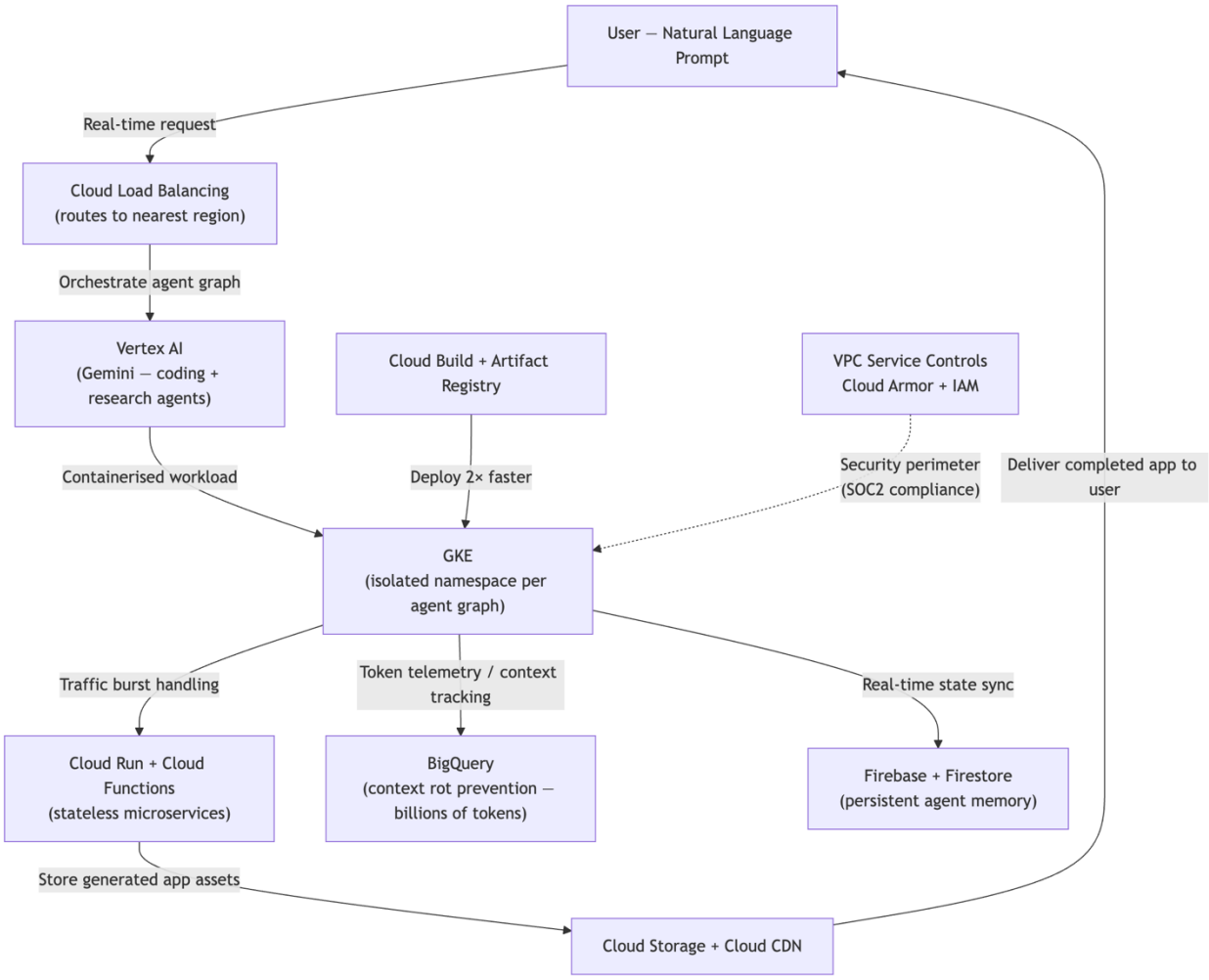

Figure 2: ChatAndBuild Google Cloud architecture — natural language prompt through Vertex AI orchestration, GKE execution, and CDN delivery to end user (Google Cloud, n.d.).

3.5. Reflection

ChatAndBuild's migration to Google Cloud exemplifies NIST's **rapid elasticity** and **on-demand self-service** characteristics (Mell & Grance, 2011) at their most extreme. Scaling a worker fleet from 10 to 100+ Compute Engine instances in minutes – autonomously, based

on load – is structurally impossible with on-premises infrastructure, where capacity must be physically provisioned days in advance (Google Cloud, n.d.). The 45% reduction in infrastructure overhead reflects measured service in practice: Google Cloud's transparent billing policy replaced opaque fixed infrastructure costs, freeing engineering capacity for product development rather than server management.

The contrast with traditional IT is direct. In the prior architecture, Christel Buchanan (Founder and CEO, ChatAndBuild) described engineers as acting as "the autoscaler" – manually provisioning, restarting, and monitoring systems, where any instability introduced latency or broken agent sessions (as cited in Google Cloud, n.d.). Cloud automation replaced this reactive human-in-the-loop model with policy-driven elasticity, eliminating the operational fragility that undermined user trust in long-lived AI agents. Remaining risks include data sovereignty for AI-generated content across multi-region deployments, and long-term vendor dependency on GCP's AI ecosystem as the platform deepens its reliance on Vertex AI and Gemini models.

4. Case Study 3 – Voith & AWS

4.1. Case Overview

ChatAndBuild is a Singapore-based AI-native startup whose platform enables anyone, from a 4-year-old designing a mathematics game to a 78-year-old building a strategy simulation, to turn natural language into working applications and AI agents instantly (Google Cloud, n.d.). Serving over 140,000 users, the company's vision of the "Intelligence-Ownership Economy" – where users own Non-Fungible Agents (NFAs) with persistent

memory, skills, and personality – placed extreme demands on infrastructure reliability and latency. The company's previous infrastructure imposed 30–70% higher latency on real-time generation, caused context rot in long-lived agents (loss of user history), and required engineers to manually provision servers days before global hackathons to prevent IP surge crashes (Google Cloud, n.d.). ChatAndBuild migrated to Google Cloud to establish a fully AI-native, elastically scalable infrastructure capable of operating at global speed.

4.2. Service Model Analysis

ChatAndBuild's architecture spans **IaaS** and **PaaS** layers. Google Compute Engine provides the raw virtual machine infrastructure (IaaS) that enables elastic scaling of the worker fleet from approximately 10 to 100+ instances during global hackathon events, handled automatically through instance groups and autoscaling policies (Google Cloud, n.d.). On top of this, a rich suite of managed PaaS services handles AI orchestration, containerized workload management, data analytics, state synchronization, and CI/CD (Continuous Integration / Continuous Development) – abstracting operational complexity so that the engineering team can focus on product development rather than infrastructure management (McHaney, 2021).

Service Model	Characteristic	Fit for Victoria Police
IaaS	Raw compute provisioned on demand; customer manages OS and runtime	Google Compute Engine – elastic worker fleetr scaling for burst demand
PaaS	Managed platform; provider handles runtime and infrastructure operations	**Primary layer** – Vertex AI, GKE, Clound Run, BigQuery, Cloud SQL, Firebase
SaaS	Fully managed software	Partial – Microsoft 365 (SharePoint, Teams, Outlook) as data and collaboration substrate

Table 4 – Service model analysis for ChatAndBuild (Google Cloud, n.d.).

4.3. Deployment Model Comparison

ChatAndBuild operates on **public cloud** across multiple Google Cloud regions. Cloud Load Balancing routes each request to the nearest available region, ensuring sub-second response times for users globally. The multi-region public cloud model was the only viable option for a startup serving over 140,000 users across time zones, where global hackathon spikes require instantly elastic capacity – and where the prior on-premises approach required engineers to manually provision servers days in advance (Google Cloud, n.d.).

Deployment Model	Elasticity	Global Reach	Latency	Fit for Victoria Police
On-premises	None – fixed servers	Single geography	High	Prior state – manual-provisioning; 30-70% higher latency; engineers as *"human autoscalers"*
Private Cloud	Limited	Single geography	Medium	No global edge; cannot serve 140,000+ users across time zones
Public Cloud (Multi-region)	High – Auto-scale 10→100+ instances in minutes	Global PoP via Cloud Load Balancing	Low – sub-second	**Adopted** – GKE + Cloud Run auto-scale to demand with zero downtime
Multi-Cloud	Highest	Highest	Lowest	Premature complexity for current startup scale

Table 5: Deployment model comparison for ChatAndBuild (Google Cloud, n.d.; Manvi & Shyam, 2021).

4.4. Deployment Model Comparison

The Google Cloud architecture forms an integrated "idea-to-app" engine where each service layer contributes to the real-time generation pipeline (Google Cloud, n.d.):

Service	Purpose	Business Outcome
Vertex AI + Gemini models	Routes prompts to specialised coding and research agents; multimodal generation	Core AI engine – 30–70% latency reduction vs previous setup
Google Kubernetes Engine (GKE)	Isolated namespaces per agent graph; manages thousands of concurrent builder sessions	Zero-downtime event scaling – 13B+ tokens processed at a global hackathon
Cloud Run + Cloud Functions	Stateless microservices that auto-spin for traffic bursts	Handles demand spikes without idle compute; no pre-provisioning required
Cloud Load Balancing	Routes every request to the nearest region	Sub-second response times globally
BigQuery	Telemetry layer – tracks billions of tokens to prevent context rot	Maintains long-term agent memory continuity and intelligence
Cloud SQL + Firestore + Firebase	Structured data management and real-time user state synchronization	Eliminates context rot; persistent, coherent agent sessions
Cloud CDN + Cloud Storage	Stores and delivers generated application assets globally	Final app delivery to end users with low latency
Cloud Build + Artifact Registry	CI/CD pipeline for continuous feature deployment	Feature delivery velocity doubled (2× faster)
VPC Service Controls + Cloud Armor + IAM	Security perimeter for regulated enterprise and government markets	SOC2 compliance; strict audit logging for enterprise and government partners

Table 6: Google Cloud services utilized by ChatAndBuild and their business outcomes (Google Cloud, n.d.).

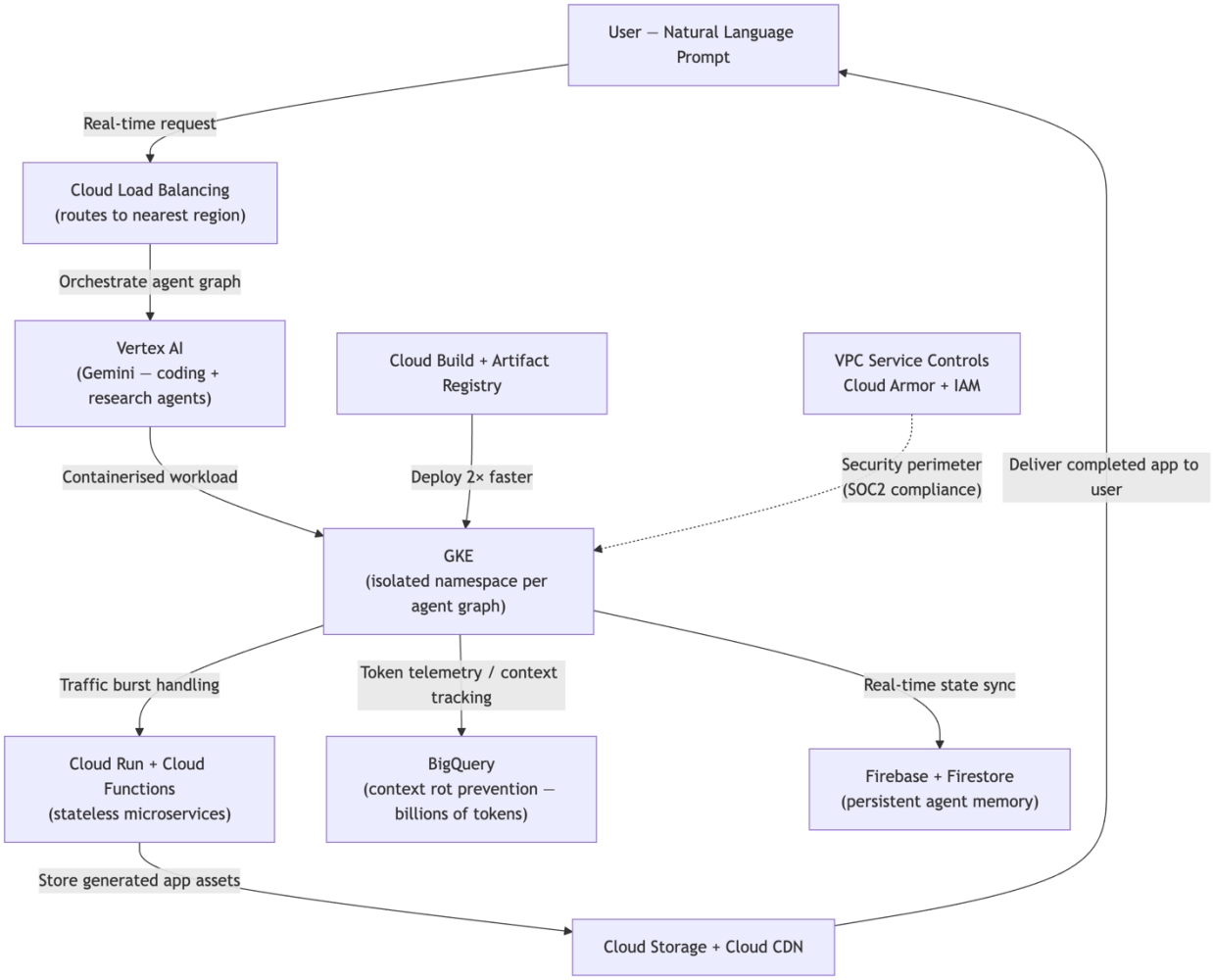

Figure 2: ChatAndBuild Google Cloud architecture — natural language prompt through Vertex AI orchestration, GKE execution, and CDN delivery to end user (Google Cloud, n.d.).

4.5. Reflection

ChatAndBuild's migration to Google Cloud exemplifies NIST's **rapid elasticity** and **on-demand self-service** characteristics (Mell & Grance, 2011) at their most extreme. Scaling a worker fleet from 10 to 100+ Compute Engine instances in minutes – autonomously, based on load – is structurally impossible with on-premises infrastructure, where capacity must be physically provisioned days in advance (Google Cloud, n.d.). The 45% reduction in infrastructure overhead reflects measured service in practice: Google Cloud's transparent billing policy replaced opaque fixed infrastructure costs, freeing engineering capacity for product development rather than server management.

The contrast with traditional IT is direct. In the prior architecture, Christel Buchanan (Founder and CEO, ChatAndBuild) described engineers as acting as "the autoscaler" – manually provisioning, restarting, and monitoring systems, where any instability introduced latency or broken agent sessions (as cited in Google Cloud, n.d.). Cloud automation replaced this reactive human-in-the-loop model with policy-driven elasticity, eliminating the operational fragility that undermined user trust in long-lived AI agents. Remaining risks include data sovereignty for AI-generated content across multi-region deployments, and long-term vendor dependency on GCP's AI ecosystem as the platform deepens its reliance on Vertex AI and Gemini models.

5. Conclusion

Cloud computing is the right strategic move for ABC: measured service, rapid elasticity, and resource pooling (Mell & Grance, 2011) translate directly into automation outcomes - scaling

without procurement delays, zero headcount growth, faster time-to-market. Recommended stack: public cloud, blended IaaS/PaaS, hybrid cost model, AWS. Managed well, ABC's infrastructure stops being a bottleneck and becomes a competitive advantage (Eliaçık, 2022).

6. Appendices

6.1. Appendix A – Cloud vs Traditional IT Comparison Table

Dimension	Traditional IT	Cloud (ABC Case)
Cost model	CAPEX heavy	OPEX elasticity
Scaling	Hardware procurement	Auto Scaling
Deployment speed	Weeks/months	Minutes
Automation	Manual ops	Policy-driven

Table 3 – Cloud vs Traditional IT Comparison.

6.2. Appendix G – Glossary of Terms

Term	Meaning
CAPEX	Capital Expenditure
OPEX	Operational Expenditure
IAM	Identity and Access Management
TCO	Total Cost of Ownership
SOA	Service-Oriented Architecture
PaaS	Platform as a Service
IaaS	Infrastructure as a Service
SaaS	Software as a Service
VPC	Virtual Private Cloud
ELB	Elastic Load Balancer
EC2	Elastic Compute Cloud
RDS	Relational Database Service
Route 53	AWS DNS Service
Auto Scaling	AWS service for automatic scaling of compute resources

Shared Responsibility	Cloud security model defining provider vs customer security obligations
Measured Service	Resource usage is monitored and billed accordingly
Rapid Elasticity	Allows resources to scale up/down automatically based on demand

Table 8 – Glossary of Terms

End of Appendix Section

7. References

Amazon Web Services. (n.d.-a). *AWS Well-Architected Framework.* https://aws.amazon.com/architecture/well-architected/

Amazon Web Services. (n.d.-b). *AWS Pricing.* https://aws.amazon.com/pricing/

Bittok, T. (2022, January 22). *Cloud total cost of ownership.* LinkedIn Pulse. https://www.linkedin.com/pulse/cloud-total-cost-ownership-theophilus-bittok-/

Carpenter, T. (2020, May 27). *AWS Certified Solutions Architect – Associate (SAA-C02) Cert Prep 1: Cloud services overview*. LinkedIn Learning. https://www.linkedin.com/learning/aws-certified-solutions-architect-associate-saa-c02-cert-prep-1-cloud-services-overview/benefits-of-cloud-computing

Eliaçık, E. (2022). *Pros and cons of cloud computing.* Dataconomy. https://dataconomy.com/2022/05/pros-and-cons-of-cloud-computing-2022/

IBM. (n.d.-a). *SaaS, PaaS, IaaS explained.* IBM Cloud. https://www.ibm.com/think/topics/iaas-paas-saas

IBM. (n.d.-b). *What is a public cloud?* IBM. https://www.ibm.com/think/topics/public-cloud

IBM. (n.d.-c). *What is private cloud?* IBM. https://www.ibm.com/think/topics/private-cloud

Linthicum, D. (2021, May 25). *Learning cloud computing: Core concepts.* LinkedIn Learning. https://www.linkedin.com/learning/learning-cloud-computing-core-concepts-13710481/

Manvi, S., & Shyam, G. K. (2021). *Cloud computing: Concepts and technologies* (Chapter 4). CRC Press. https://learning-oreilly-com.torrens.idm.oclc.org/library/view/cloud-computing/9781000338058/

McHaney, R. (2021). *Cloud technologies: An overview of cloud computing technologies for managers.* Wiley. https://ieeexplore-ieee-org.torrens.idm.oclc.org/servlet/opac?bknumber=9820907

Mell, P., & Grance, T. (2011). *The NIST definition of cloud computing* (Special Publication 800-145). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-145

Naved, M., Sanchez, D. T., Dela Cruz, A. P., Jr., Peconcillo, L. B., Jr., Peteros, E. D., & Tenerife, J. J. L. (2022). Identifying the role of cloud computing technology in management of educational institutions. *Materials Today: Proceedings, 51*(8), 2309–2312. https://doi.org/10.1016/j.matpr.2021.11.414

Nishimura, H. (2022, August 30). *Introduction to AWS for non-engineers: 1 cloud concepts*. LinkedIn Learning. https://www.linkedin.com/learning/introduction-to-aws-for-non-engineers-1-cloud-concepts-2/

Shore, J. (2020). *Cybersecurity with cloud computing: Service models*. LinkedIn Learning. https://www.linkedin.com/learning/cybersecurity-with-cloud-computing-2/